Corporación Universitaria Iberoamericana

Inteligencia artificial

Actividad 2 — Sistema inteligente para la búsqueda de rutas

Estudiante: Laura Lorena Forero Romero
Docente: Sandra Bautista
Grupo: 24082026_C1_202634

ACTIVIDAD 2 – SISTEMA INTELIGENTE PARA LA BÚSQUEDA DE RUTAS

Introducción

En esta actividad desarrollé un programa en Python que busca la ruta de menor tiempo estimado entre dos estaciones de una red de transporte masivo. Lo hice para aplicar la representación del conocimiento mediante hechos y reglas, y para comprender cómo funciona una búsqueda heurística. Elegí como contexto Bogotá, pero construí una red pequeña y simulada: las estaciones, las líneas, los trayectos y sus tiempos no describen el funcionamiento real de TransMilenio.

Mi intención fue pasar de una pregunta sencilla, "¿cómo llego desde una estación hasta otra?", a un procedimiento que pudiera revisar las conexiones disponibles, sumar los tiempos y comparar diferentes posibilidades. También preparé pruebas para observar qué ocurre cuando la estación no existe, el destino está desconectado o cambia el costo de un transbordo.

1. Objetivo y alcance

Mi objetivo fue implementar un prototipo reproducible que, a partir de una base de conocimiento con estaciones y conexiones, encuentre un camino de costo mínimo. Para esta actividad definí "mejor ruta" como la de menor suma de minutos de recorrido y penalizaciones por cambio de línea. El programa no intenta calcular tarifas, predecir la llegada de buses ni recomendar desplazamientos reales.

Trabajé con Python y su biblioteca estándar, por lo que no es necesario instalar paquetes adicionales para ejecutar el algoritmo o sus pruebas. El código fuente principal está en rutas.py; las pruebas automáticas están en tests/test_rutas.py y las instrucciones de uso se encuentran en INSTRUCCIONES.txt.

2. Representación del conocimiento

Primero registré los hechos en dos estructuras. ESTACIONES almacena los nombres y unas coordenadas abstractas que sirven para orientar la búsqueda. CONEXIONES reúne pares de estaciones vecinas, los minutos necesarios para viajar entre ellas y la línea a la que pertenece cada conexión. Por ejemplo, entre Portal y Norte hay una conexión Azul de cuatro minutos. El programa convierte cada conexión en dos sentidos para esta red académica.

A partir de esos hechos apliqué tres reglas. La primera permite pasar de una estación a otra solo cuando existe una conexión registrada. La segunda añade dos minutos al cambiar de línea, pero no cobra ese tiempo al abordar por primera vez. La tercera indica que, si después de explorar las conexiones no se llega al destino,

el resultado debe ser “sin ruta”. Estas reglas están expresadas como condiciones e instrucciones dentro de la función mejor_ruta.

Representé cada estado como una pareja (estación, línea actual). Tomé esta decisión porque estar en la misma estación no siempre implica el mismo costo futuro: si llego por la línea Azul y el siguiente tramo también es Azul, no hay transbordo; si llego por otra línea, sí debo sumarlo.

3. Estrategia de búsqueda

Utilicé A*, que organiza los estados pendientes mediante f(n) = g(n) + h(n). En mi programa, g(n) es el costo acumulado de recorrido y transbordos. La función h(n) estima el costo restante mediante la distancia euclidiana entre las coordenadas abstractas de la estación actual y el destino. La cola de prioridad permite revisar primero los estados con menor valor estimado.

En la red que construí, cada conexión tiene un tiempo mayor o igual que la distancia geométrica entre sus extremos, y un transbordo nunca reduce el costo. Por eso la estimación no supera el costo mínimo pendiente y A* puede hallar una solución de menor costo en este ejemplo. Si alguien modifica la red, tendrá que comprobar de nuevo esa condición. Una alternativa segura sería usar h(n) = 0, caso en el que la búsqueda equivale al algoritmo de Dijkstra para costos no negativos.

Cuando el algoritmo llega al destino, recupera los estados anteriores para presentar la secuencia de estaciones, las líneas utilizadas y los minutos calculados. Si no encuentra el destino, devuelve un resultado vacío que la interfaz expresa como ausencia de ruta.

4. Instrucciones para ejecutar el proyecto

Guardé todos los archivos en una misma carpeta. Desde una terminal ubicada allí, puedo ver las estaciones, ejecutar una consulta y lanzar las pruebas con los siguientes comandos:

```
python rutas.py --estaciones
python rutas.py Portal Sur
python rutas.py Portal Centro
python -m unittest discover -s tests -v
```

En Windows también puedo utilizar py si ese es el comando configurado para Python. El programa exige un origen y un destino escritos exactamente como aparecen en la lista de estaciones. Para ejecutarlo necesito Python 3.10 o una versión posterior.

5. Resultados del caso principal

Al buscar una ruta de Portal a Sur obtuve: Portal - Norte - Occidente - Parque - Hospital - Sur. Las líneas son Azul para el primer tramo y Verde para los cuatro siguientes. El programa informa 18 minutos simulados. Puedo verificar el cálculo sumando 4 + 3 + 3 + 3 + 3 = 16 minutos de recorrido y 2 minutos por el cambio de Azul a Verde.

Comparé ese resultado con otra posibilidad: Portal - Norte - Centro - Universidad - Hospital - Sur. Sus recorridos suman 18 minutos y sus dos cambios de línea añaden cuatro minutos, para un total de 22. Esta comparación ayuda a entender por qué el itinerario obtenido cuesta menos dentro de la red definida; no es una comparación de rutas reales de Bogotá.

6. Pruebas realizadas y evidencia

Escribí seis pruebas automáticas con unittest. Para cada una definí una entrada, un comportamiento esperado y una afirmación que el programa debe satisfacer. Volví a ejecutar la suite después de preparar este documento. El resultado final fue “Ran 6 tests” y “OK”. A continuación explico cada prueba por separado.

Prueba 1. Ruta directa sin cambio de línea

Busqué la ruta desde Portal hasta Centro. Esperaba que pasara por Norte usando Azul en los dos tramos. El programa devolvió Portal - Norte - Centro y ocho minutos, resultado que coincide con 4 + 4. Esta prueba confirma que se pueden encadenar conexiones y sumar sus tiempos sin añadir un transbordo innecesario.

Prueba 2. Ruta con transbordo y selección del menor costo

Busqué la ruta desde Portal hasta Sur. Esperaba el trayecto Portal - Norte - Occidente - Parque - Hospital - Sur, con 18 minutos. La prueba comprobó tanto la secuencia de estaciones como las líneas Azul, Verde, Verde, Verde y Verde. El resultado coincidió con lo esperado y mostró que el cambio de línea se suma una sola vez.

Prueba 3. Origen y destino iguales

Elegí Centro como origen y como destino. En esta situación no hace falta recorrer una conexión ni cambiar de línea. El algoritmo devolvió Centro con un tiempo de cero minutos. La prueba comprueba que el programa resuelve correctamente este caso límite.

Prueba 4. Estación que no existe

Escribí Inventada como estación de origen y Centro como destino. El programa generó un error controlado que informa que la estación es desconocida. Con esto verifiqué que una entrada fuera de la base de conocimiento no produce una ruta ficticia.

Prueba 5. Destino inalcanzable

Preparé temporalmente un grafo en el que Sur no tiene conexiones y luego pedí una ruta desde Portal. El algoritmo terminó la exploración y devolvió None, que significa que no encontró un camino. Esta prueba permite distinguir un destino registrado pero aislado de una estación inexistente.

Prueba 6. Cambio en la penalización del transbordo

Repetí la búsqueda desde Portal hasta Sur con una penalización de cero minutos al cambiar de línea. El costo calculado pasó de 18 a 16 minutos. De esta manera comprobé que la regla de transbordo realmente interviene en el cálculo y que puedo modificar ese parámetro sin cambiar las conexiones.

En la ejecución de unittest las seis pruebas se identifican como test_desconocida, test_inalcanzable, test_mismo_origen_destino, test_penalizacion, test_ruta_directa y test_transbordo_y_optimo. Cada una terminó con “ok”. Para repetir la evidencia basta con ejecutar el último comando de la sección 4 desde la carpeta del proyecto.

7. Limitaciones y posibles mejoras

La principal limitación de mi trabajo es que los datos fueron creados para aprender. No incorporé recorridos oficiales, sentidos de operación, frecuencias, estaciones cerradas, congestión ni condiciones de accesibilidad. Por esa razón, los minutos obtenidos solo permiten verificar el comportamiento del algoritmo. No utilizaría este programa para decidir un viaje real sin reemplazar y validar antes toda la base de conocimiento.

Como mejora, podría cargar conexiones desde un archivo revisado, permitir diferentes criterios de búsqueda y mostrar en pantalla cada transbordo de manera más clara. También tendría que comprobar que la heurística siga siendo válida después de introducir nuevas estaciones y tiempos.

Conclusiones

Con esta actividad comprendí que una base de conocimiento no tiene que limitarse a una lista de datos: también necesita reglas que indiquen cómo utilizar esos datos. En mi caso, las reglas de conexión y transbordo transformaron las estaciones en un problema que el programa puede resolver y explicar.

Las seis pruebas me ayudaron a revisar el resultado principal y situaciones que podrían pasarse por alto, como pedir una estación desconocida o un destino sin conexión. Observé que el costo de una ruta depende de los tiempos de recorrido y de la forma en que se representa cada cambio de línea.

Finalmente, pude relacionar los conceptos de búsqueda heurística con una implementación concreta en Python. El prototipo responde bien para la red simulada, y aprendí que obtener una ruta útil fuera del ejercicio requiere datos confiables y actualizados, además de un algoritmo correcto.

Enlaces de entrega

El código fuente y las instrucciones de ejecución están en el repositorio:
https://github.com/lauraromeroforero/InteligenciaArtificialPython.git

La carpeta compartida indicada para el documento y el video es:
https://drive.google.com/drive/folders/1C9uXlLQ2zHUr_G8TTDZYv0uYEjdvPgQk

Bibliografía

Benítez, R. (2014). Inteligencia artificial avanzada. Editorial UOC. Capítulos 2, 3 y 9 sugeridos para esta actividad.

Corporación Universitaria Iberoamericana. (s. f.). Bibliografía de la unidad de Inteligencia Artificial. Material del curso. https://campusvirtual.ibero.edu.co/repositorio/Cursos-Matriz/Ingenieria/Ingenieria-software/Inteligencia-Artificial/Inteligencia_artificial/index.html

Anexo: representación explícita de las reglas lógicas

La siguiente notación expresa los hechos y reglas que implementa el programa en Python. No requiere Prolog ni constituye un intérprete adicional. Las variables A, B, L y T representan estaciones, línea y minutos; P representa la penalización.

Hechos de ejemplo

```
estacion(Portal). estacion(Norte).
conexion(Portal, Norte, 4, Azul).
conexion(Norte, Occidente, 3, Verde).
```

R1. Conexión bidireccional

```
conexion(A, B, T, L) => paso(A, B, T, L).
conexion(A, B, T, L) => paso(B, A, T, L).
```

R2. Primer abordaje

```
linea_actual = ninguna => costo_transbordo = 0.
```

R3. Permanencia en la línea

```
linea_actual = L => costo_transbordo = 0.
```

R4. Cambio de línea

```
linea_actual != ninguna AND linea_actual != L
  => costo_transbordo = P.
```

R5. Transición válida

```
paso(A, B, T, L) AND estado_actual = (A, linea_actual)
  => estado_siguiente = (B, L)
     AND costo_siguiente = costo_actual + T + costo_transbordo.
```

R6. Meta y ausencia de solución

```
estacion_actual = destino => reconstruir_ruta.
cola_pendiente = vacia AND destino_no_alcanzado => sin_ruta.
```

Relación con el código

crear_grafo convierte los hechos de CONEXIONES en pasos en ambos sentidos (R1). mejor_ruta calcula la penalización (R2–R4), actualiza estados y costos (R5) y devuelve una Ruta o None (R6). El criterio de mínimo costo se resuelve con A*, no con una recomendación de transporte real.

Validación actual del proyecto

```
test_desconocida (test_rutas.PruebasRutas.test_desconocida) ... ok
test_inalcanzable (test_rutas.PruebasRutas.test_inalcanzable) ... ok
test_mismo_origen_destino (test_rutas.PruebasRutas.test_mismo_origen_destino) ... ok
test_penalizacion (test_rutas.PruebasRutas.test_penalizacion) ... ok
test_ruta_directa (test_rutas.PruebasRutas.test_ruta_directa) ... ok
test_transbordo_y_optimo (test_rutas.PruebasRutas.test_transbordo_y_optimo) ... ok

----------------------------------------------------------------------
Ran 6 tests in 0.001s

OK
```

Consultas verificadas

```
Ruta: Portal - Norte - Occidente - Parque - Hospital - Sur
Líneas: Azul, Verde, Verde, Verde, Verde
Tiempo simulado: 18 minutos
Ruta: Portal - Norte - Centro
Líneas: Azul, Azul
Tiempo simulado: 8 minutos
Ruta: Portal - Norte - Occidente - Parque - Hospital - Sur
Líneas: Azul, Verde, Verde, Verde, Verde
Tiempo simulado: 16 minutos
```